

Here is the complete architectural overview of the **Callog (AllSet CRM)** project, organized hierarchically from the root directory down to every individual file with its responsibilities and implementation details.

Architecture Overview

```
callog/
├── Gradle & Configs                                # Build configuration, Gradle wrapper, properties
├── AGENTS.md & README.md                          # Workspace rules, architecture guide, project overview
├── app/
│   ├── src/main/AndroidManifest.xml# Permissions, receivers, deep-link intent filters
│   └── src/main/java/com/example/callog/
│       ├── CallVaultApp.kt                        # Application class (@HiltAndroidApp), WorkManager setup
│       ├── MainActivity.kt                        # Single Activity, Intent handling, Compose Root
│       ├── core/                                  # Utilities, diagnostics, logging, connectivity
│       ├── data/                                  # Data layer (Room DAOs, Entities, Providers, Workers, Repo)
│       ├── di/                                    # Dagger Hilt dependency injection modules
│       ├── domain/                                # Business logic, models, repository interfaces, use cases
│       ├── presentation/                          # Jetpack Compose UI, Navigation 3, ViewModels, Themes
│       └── sim/                                    # Dual-SIM detection, routing, and management
```

1. ⚙️ Root Configuration & Project Files

File Name	Description & Code Details
build.gradle.kts	Root Gradle Build Script: Defines plugins applied across all submodules (Android Application 8.7.3 ,

File Name	Description & Code Details
	Kotlin Android 2.1.0, Dagger Hilt 2.51.1, Kotlin Serialization 2.1.0, KSP 2.1.0-1.0.29).
settings.gradle.kts	Gradle Settings: Configures plugin management repositories (Google, MavenCentral, Gradle Plugin Portal) and includes the :app module.
gradle.properties	JVM & Build Properties: Allocates JVM heap memory (-Xmx2048m), enables AndroidX (android.useAndroidX=true), and configures Kotlin compilation optimizations.
gradlew / gradlew.bat	Gradle Wrapper: Unix/Bash and Windows batch executables ensuring reproducible Gradle runtime execution.
AGENTS.md	Agent Guidelines & Architectural Rules: Comprehensive rules covering Clean Architecture layers, Kotlin Coroutines dispatchers (Dispatchers.IO for DB/network, Dispatchers.Main for UI), Compose UDF patterns, Room migration rules, and build/test commands.
README.md	Project Documentation: High-level overview of features, setup instructions, architecture diagrams, and cloud integrations (Firebase Firestore, Cloud Storage, Supabase PostgreSQL).
app/build.gradle.kts	App Module Build Script: Configures compileSdk = 35, minSdk = 26, targetSdk = 35, Jetpack Compose BOM 2025.09.00, Material 3, Hilt, Room 2.6.1, Retrofit, Firebase Firestore/Storage, Supabase, and JUnit testing dependencies.
app/src/main/AndroidManifest.xml	Android Manifest: Declares runtime permissions (READ_CALL_LOG, READ_CONTACTS, READ_PHONE_STATE, READ_MEDIA_AUDIO, INTERNET), registers CallVaultApp, MainActivity, CallReceiver broadcast receiver, and registers deep-link <intent-filter> for allset:// and callog:// URI schemes.

2. 🚀 Application Root & Core Layer (core/)

File Name	Description & Code Details
CallVaultApp.kt	Application Class: Annotated with <code>@HiltAndroidApp</code> . Implements <code>Configuration.Provider</code> to inject <code>HiltWorkerFactory</code> for dependency injection in background <code>WorkManager</code> jobs (<code>SyncWorker</code> , <code>UploadRecordingWorker</code>).
MainActivity.kt	Main ComponentActivity: Annotated with <code>@AndroidEntryPoint</code> . Sets up Edge-to-Edge Compose display, intercepts incoming deep links (<code>Intent.ACTION_VIEW</code>), passes parsed URIs to Navigation 3 / AllSet hosts, and hosts <code>NavGraph</code> .
core/config/SupabaseDefaults.kt	Supabase Constants: Holds default Supabase endpoint URLs, table names (<code>sales_calls</code>), and fallback configuration schemas.
core/constants/Constants.kt	Global Constants: Storage keys, shared preferences identifiers, channel notification IDs, sync interval timers, and audio MIME types.
core/diagnostics/DeveloperLogger.kt	In-Memory Diagnostic Logger: Thread-safe circular buffer logging recording matching heuristics, SIM detection events, and sync results for display in <code>SyncLogsScreen</code> and <code>RecordingDiagnosticsScreen</code> .
core/extensions/Extensions.kt	Kotlin Utility Extensions: Formatting epoch timestamps to readable dates, phone number normalization (stripping non-numeric characters), and duration formatting (<code>MM:SS</code>).
core/utils/ConnectivityService.kt	Network Monitor: Uses <code>ConnectivityManager.NetworkCallback</code> to emit reactive <code>Flow<Boolean></code> network status for offline-first sync triggering.

3. Data Layer (data/)

3.1 Local Database (Room) — data/local/

File Name	Description & Code Details
local/database/CallDatabase.kt & CallVaultDatabase.kt	Room Database: Defines schema versioning, entities (<code>CallEntity</code> , <code>SalesCallEntity</code> , <code>RecordingEntity</code> , <code>SyncLogEntity</code> , <code>TracebackEntity</code> , <code>ReminderEntity</code>), and provides abstract DAO accessor methods.
local/database/Converters.kt	Room Type Converters: Serializes/deserializes complex types (e.g. <code>List<String></code> , <code>UploadStatus</code> , <code>MatchStatus</code> , <code>Date</code> objects) to/from SQLite compatible types.
local/entity/CallEntity.kt	Call Entity: Primary table storing call logs (number, name, call type, timestamp, duration, SIM slot, notes, tags, isFavorite, recording path).
local/entity/SalesCallEntity.kt	Sales Call Entity: Structured CRM call records with deal size, lead status, customer name, and follow-up flags.
local/entity/RecordingEntity.kt & RecordingLogEntity.kt	Recording Entities: Track discovered audio files, file size, matched caller ID, duration, and cloud upload state (<code>PENDING</code> , <code>UPLOADING</code> , <code>UPLOADED</code> , <code>FAILED</code>).
local/entity/SyncLogEntity.kt & TracebackEntity.kt	Sync & Diagnostics Entities: Record background sync attempts, payload counts, errors, stack traces, and latency.
local/entity/ReminderEntity.kt	Reminder Entity: Scheduled follow-ups linked to specific call IDs with due timestamps and completion flags.
local/dao/CallDao.kt	Call DAO: Reactive Flow queries (<code>getRecentCallsFlow</code> , <code>getFavoriteCallsFlow</code> , <code>getRecordingCallsFlow</code>), tag updates, note edits, and batch insertions.

File Name	Description & Code Details
local/dao/SalesCallDao.kt	Sales Call DAO: CRUD operations for sales CRM records, querying unsynced sales calls.
local/dao/RecordingDao.kt & RecordingLogDao.kt	Recording DAOs: Queries for pending uploads, matched recordings, and file path verification.
local/dao/SyncLogDao.kt & TracebackDao.kt	Diagnostic DAOs: CRUD for sync history audit trails and runtime traceback logs.
local/dao/ReminderDao.kt	Reminder DAO: Queries pending reminders joined with call metadata (ReminderWithCall).

3.2 Providers, Receivers, Remote, Repositories, Workers

File Name	Description & Code Details
data/provider/CallLogProvider.kt	System Call Log Scanner: Queries Android's CallLog.Calls ContentResolver, extracts duration, caller number, call type, SIM subscription ID, and maps to domain models.
data/provider/ContactsProvider.kt	System Contacts Scanner: Queries ContactsContract.CommonDataKinds.Phone, retrieves contact ID, display name, normalized phone numbers, and contact photo URIs.
data/provider/RecordingScanner.kt & RecordingParser.kt	Audio Recording Scanner & Parser: Traverses device storage directories (MIUI/Xiaomi, Samsung, ColorOS, Pixel), extracts timestamps and caller metadata from filenames/tags using fuzzy heuristic matching.
data/receiver/CallReceiver.kt	Broadcast Receiver: Listens for TelephonyManager.ACTION_PHONE_STATE_CHANGED, detects incoming/outgoing/ended call events,

File Name	Description & Code Details
	and triggers immediate background sync jobs.
data/remote/FirestoreService.kt	Firestore Remote Client: Manages Firestore authentication and writes call log documents into user-specific Firestore collections.
data/remote/SupabaseService.kt	Supabase Remote Client: Interacts with Supabase PostgREST endpoints to upsert structured sales records and CRM data.
data/remote/model/SupabaseSalesCall.kt	Supabase DTO: Serialization model mapping Kotlin properties to Postgres columns (call_id, caller_number, duration_seconds, deal_stage, created_at).
data/repository/CallRepositoryImpl.kt	Call Repository Implementation: Coordinates local Room cache with CallLogProvider, manages reactive streams, and updates tags/notes.
data/repository/FirestoreRepositoryImpl.kt	Firestore Repository Implementation: Implements domain interface for syncing calls to Google Cloud Firestore.
data/repository/RecordingRepositoryImpl.kt	Recording Repository Implementation: Implements discovery, local audio playback matching, and upload queue management.
data/repository/SyncRepositoryImpl.kt	Sync Repository Implementation: Logs sync transactions, tracks network-awareness status, and computes sync success metrics.
data/service/SyncManagerImpl.kt & UploadServiceImpl.kt	Sync & Upload Coordinators: Coordinate batch synchronization flows and upload file streams to Firebase Cloud Storage.

File Name	Description & Code Details
data/worker/SyncWorker.kt	WorkManager Background Sync Worker: Executes periodic and on-demand synchronization of calls with exponential backoff and network constraints.
data/worker/UploadRecordingWorker.kt	WorkManager Audio Upload Worker: Background worker for uploading large .m4a / .mp3 audio files to Firebase Cloud Storage.

4. 📁 Dependency Injection (di/)

File Name	Description & Code Details
di/AppModule.kt	Application Module: Provides application Context , ContentResolver , SharedPreferences , and global Coroutine dispatchers (Dispatchers.IO , Dispatchers.Default).
di/DatabaseModule.kt	Database Module: Provides singleton instances of CallDatabase , CallVaultDatabase , and all individual DAOs (CallDao , SalesCallDao , RecordingDao , SyncLogDao , etc.).
di/FirestoreModule.kt	Firestore Module: Provides FirebaseFirestore and FirebaseStorage singleton instances.
di/NetworkModule.kt	Network Module: Provides OkHttpClient , Retrofit converters, and ConnectivityService .
di/ServiceModule.kt	Service Binds Module: Binds domain repository and service interfaces (CallRepository , SyncRepository , SyncManager , UploadService) to their concrete data layer implementations.

5. 🧠 Domain Layer (domain/)

File Name	Description & Code Details
domain/model/Call.kt & CallLogEntry.kt	Domain Call Models: Immutable domain models representing call interactions (name, number, duration, timestamp, SIM slot, recording URL, tags).
domain/model/FirebaseConfig.kt & SupabaseConfig.kt	Configuration Domain Models: Strongly typed credentials and configurations for cloud backends.
domain/repository/CallRepository.kt	Call Repository Interface: Pure business contract for call log retrieval, notes/tag modifications, and favorite toggling.
domain/repository/FirestoreRepository.kt & RecordingRepository.kt	Remote & Recording Repository Interfaces: Contracts for Firestore sync and recording management.
domain/repository/SyncRepository.kt	Sync Repository Interface: Contract for logging and retrieving sync diagnostics.
domain/service/SyncManager.kt & UploadService.kt	Service Contracts: Interfaces governing sync orchestration and file uploads.
domain/usecase/SyncPendingCallsUseCase.kt & UseCases.kt	Atomic Business Use Cases: Encapsulate specific workflows (e.g. scanning new calls, matching audio recordings, and uploading pending records).

6. Presentation Layer (presentation/)

6.1 Navigation 3 & AllSet Architecture — `presentation/navigation3/`

File Name	Description & Code Details
AllSetNavKey.kt	Type-Safe Navigation Keys: Defines <code>@Serializable</code> keys for top-level CRM destinations (Home, Contacts, Tasks, Meetings, Settings), Contacts search & detail sub-sections (Calls,

File Name	Description & Code Details
	WhatsApp, Meetings, Tasks, Feedback, InteractionHistory), modal flows (EditContact, AddFeedback), and Auth flows.
AllSetBackStack.kt	Observable Backstack & Result Passing: User-owned SnapshotStateList<AllSetNavKey> with push, pop, popTo, resetToRoot, and integrated AllSetResultManager (setResult, consumeResult).
AllSetMultiBackStack.kt	Multi-Backstack State Manager: Maintains isolated backstacks per CRM tab, preserving nested navigation and scroll state when switching between tabs.
AllSetNavDisplay.kt	NavDisplay Transition Host: Declarative Compose container rendering the active key with slide/fade animations and system back interception (BackHandler).
AllSetDeepLinkParser.kt	Deep Link Parser: Pure JVM & Android URI parser mapping allset://contact/{id}, allset://whatsapp/{id}, allset://task/{id}, and allset://meeting/{id} directly into typed AllSetNavKey instances.
AllSetAuthRouter.kt	Conditional Auth Router: Automatically routes between Login/Auth stack and Main CRM stack based on reactive AllSetAuthState (Unauthenticated VS Authenticated).
AdaptiveContactsPane.kt	Adaptive Large-Screen Layout: Dual-pane responsive layout displaying Contact List and Contact Details side-by-side on tablets/foldables (windowWidthDp >= 600.dp).
AllSetCrmHost.kt	AllSet CRM Host Container: Coordinates top-level Navigation 3 tab bar, sub-screens, and result-passing handlers.
Nav3Key.kt to CrmNav3Container.kt	Core Nav3 Primitives: Generic Navigation 3 building blocks providing deep link handling, multi-backstack state, and animated transition containers.

6.2 Screens — presentation/screens/

File Name	Description & Code Details
contacts/ContactsScreen.kt	Contacts List Screen: Renders contact list with search bar, contact avatars, phone numbers, and triggers navigation on click.
contacts/ContactDetailsScreen.kt	Canonical Contact Details Screen: Driven by canonical <code>contactId</code> , displays customer metadata, sub-section shortcuts (Calls, WhatsApp, Meetings, Tasks, Feedback), and call history.
contacts/GlobalSearchScreen.kt	Global Search & Results Screens: Real-time query filtering across contacts with highlighted matches.
contacts/ContactSubScreens.kt	Contact Sub-Flows: <code>WhatsAppScreen</code> , <code>EditContactScreen</code> (returns edited result), and <code>AddFeedbackScreen</code> (5-star rating + notes result passing).
tasks/TaskScreens.kt	Task Management Flows: <code>TaskListScreen</code> , <code>CreateTaskScreen</code> (with result callback), and <code>TaskDetailsScreen</code> .
meetings/MeetingScreens.kt	Meeting Flows: <code>MeetingListScreen</code> , <code>ScheduleMeetingScreen</code> , and <code>MeetingDetailsScreen</code> .
auth/AuthScreens.kt	Authentication Screens: <code>LoginScreen</code> , <code>RegisterScreen</code> , <code>ForgotPasswordScreen</code> , and <code>OtpVerificationScreen</code> .
home/HomeScreen.kt & dashboard/DashboardScreen.kt	Home & Dashboard Screens: Quick stats overview, recent interactions, sync health status widget, and quick actions.
logs/CallLogsScreen.kt	Call Logs Feed: Main list of incoming, outgoing, and missed calls with audio player controls for matched recordings.

File Name	Description & Code Details
details/CallDetailsScreen.kt & DetailsScreen.kt	Call Detail Screens: Displays specific call metadata, duration, SIM slot used, recording waveform/playback, tags, and editable notes.
recordings/RecordingManagerScreen.kt	Recording Manager: Lists discovered call recordings, match confidence indicators, and manual matching controls.
analytics/AnalyticsScreen.kt	Call Analytics: Visual charts for call volume, duration metrics, talk-time per SIM, and peak interaction hours.
favorites/FavoritesScreen.kt	Favorites Screen: Filtered feed of starred calls and key client interactions.
settings/SettingsScreen.kt	Settings Screen: Cloud backend credentials configuration (Firebase/Supabase), auto-sync toggles, and business SIM selection.
developer/DeveloperDashboardScreen.kt , RecordingDiagnosticsScreen.kt , SyncLogsScreen.kt	Developer Diagnostic Suite: Real-time diagnostic monitors for audio scanning heuristics, background sync attempts, and live system tracebacks.
permission/PermissionScreen.kt , onboarding/OnboardingScreen.kt , splash/SplashScreen.kt	Onboarding & Permissions: Runtime permission rationale dialogs, introductory walkthroughs, and animated splash screen.

6.3 Components, ViewModels, Theme & Legacy Navigation

File Name	Description & Code Details
components/CallCard.kt	Call Interaction Card: Reusable card with caller avatar, call type badge (incoming/outgoing/missed), SIM indicator, duration, and audio play button.

File Name	Description & Code Details
components/BusinessSimWizardDialog.kt	Dual-SIM Configuration Wizard: Interactive dialog to select which SIM slot is designated for business call syncing.
components/CommonComponents.kt , EmptyState.kt , LoadingView.kt , PermissionDialog.kt , SearchBar.kt	Design System Components: Custom search bars, empty state placeholders, loading spinners, and permission dialogs.
viewmodel/ContactDetailsViewModel.kt	Contact Details ViewModel: Receives canonical <code>contactId: String</code> , queries repository for details and call logs, and manages UI state.
viewmodel/CallViewModel.kt & HomeViewModel.kt	Main & Home ViewModels: Manage call log state flows, search query filtering, manual sync triggers, and audio playback state.
viewmodel/AnalyticsViewModel.kt , FavoritesViewModel.kt , SettingsViewModel.kt	Feature ViewModels: Manage analytics aggregations, starred calls, and app settings state flows.
theme/Color.kt , Theme.kt , Type.kt	Material 3 Design System: Custom dark/light color schemes, slate/indigo palettes, typography scales, and shapes.
navigation/BottomNavigationBar.kt , NavGraph.kt , Screen.kt	Navigation Graph: Bridge routing between existing screens and the new Navigation 3 CRM host.

7. Dual-SIM Management (sim/)

File Name	Description & Code Details
sim/SimInfo.kt	SIM Data Model: Represents SIM slot index (0 or 1), subscription ID, carrier name, country ISO, display name, and icon tint.

File Name	Description & Code Details
sim/SimManager.kt	Hardware SIM Manager: Interacts with Android's <code>SubscriptionManager</code> , detects active SIM subscriptions, identifies dual-SIM hardware, and maps call logs to physical SIM slots.
sim/SimRepository.kt	SIM Preferences Repository: Persists user selection for which SIM is designated for business syncing.

8. Unit Test Suites (app/src/test/)

File Name	Description & Code Details
AllSetBackNavigationTest.kt	Back Navigation Unit Tests: Tests <code>push</code> , <code>pop</code> , <code>popTo</code> , and <code>resetToRoot</code> backstack manipulations.
AllSetDeepLinkTest.kt	Deep Link Parser Tests: Tests URI routing (<code>allset://contact/{id}</code> , <code>whatsapp/{id}</code> , <code>task/{id}</code> , <code>meeting/{id}</code>).
AllSetAuthRoutingTest.kt	Conditional Auth Tests: Tests navigation state transitions between <code>Unauthenticated</code> and <code>Authenticated</code> .
AllSetContactNavigationTest.kt	Canonical ContactId Tests: Tests contact navigation using canonical IDs and sub-screen transitions.
AllSetMultiBackStackTest.kt	Multi-Backstack Isolation Tests: Tests state preservation across tab switches (<code>Contacts</code> → <code>Meetings</code> → <code>Tasks</code> → <code>Contacts</code>).
AllSetResultPassingTest.kt	Result Passing Tests: Tests <code>AllSetResultManager</code> result propagation for <code>EditContact</code> , <code>AddFeedback</code> , and <code>CreateTask</code> .